Universidad del Valle de Guatemala

Facultad de Ingeniería

Computación Paralela y Distribuida - Semestre 2, 2026



Proyecto 02

Búsqueda de claves AES-128 con Open MPI

Distribución estática y búsqueda por familias

Integrantes:

Daniel Estrada - 20853

Andre Pivaral - 23574

Sebastian Estrada - 21405

Guatemala, 28 de octubre de 2026

Índice

1. Introducción y fundamentos	3
2. Archivos, cifrado y validación	4
3. Estrategia base: lotes estáticos	5
4. Alternativa: familias y avisos	6
5. Método y resultados	7
6. Análisis del rendimiento	8
7. Discusión y conclusiones	9
Referencias bibliográficas	10
Anexo 1. Funciones AES y archivos	11
Anexo 1. Funciones de búsqueda y MPI	12
Anexo 1. Familias y herramientas	13
Anexo 1. Librerías y comunicación	14
Anexo 2. Compilación y pruebas	15
Anexo 2. Archivos	16
Anexo 2. Búsqueda del caso a)	17
Anexo 2. Búsqueda del caso b)	18
Anexo 2. Búsqueda del caso c)	19
Anexo 2. Muestras y reproducción	20

Resumen

Se implementaron el cifrado y descifrado de archivos, una búsqueda secuencial y dos versiones MPI. La base redujo el tiempo frente a la secuencial en las búsquedas completadas. La alternativa encontró las claves grandes al priorizar potencias de dos y sus vecinos, pero fue más lenta en la clave pequeña y en el control sin familias.

El informe reúne los resultados de 60 ejecuciones, las pruebas funcionales y las siete capturas de evidencia. Los recorridos exhaustivos de las claves grandes terminaron por límite de tiempo; sus estimaciones se presentan aparte.

1. Introducción y fundamentos

El proyecto estudia cómo el reparto de trabajo y el orden de búsqueda afectan el tiempo necesario para recuperar una clave AES-128. Parte del Laboratorio 03 y cumple las actividades del enunciado: procesar archivos, probar tres claves con cuatro procesos MPI y comparar una estrategia base con una alternativa [7].

Objetivo y alcance

Implementar y evaluar una búsqueda paralela que identifique la clave mediante la frase «es una prueba de». Se comparan tiempo, intentos y speedup frente a una referencia secuencial. El experimento usa enteros de hasta 64 bits convertidos en claves de 16 bytes; estudia ese subconjunto, sin recorrer las 2^128 claves de AES-128.

Conceptos necesarios

Concepto	Aplicación en el proyecto
AES-128	Cifrador simétrico con bloques y clave de 128 bits [1]. Los ocho bytes iniciales de la clave se dejan en cero; los últimos ocho contienen el entero en big endian.
ECB y PKCS#7	ECB procesa cada bloque por separado [2]. El relleno completa el último bloque y añade un bloque si la longitud ya es múltiplo de 16 [3].
Búsqueda por fuerza bruta	Cada candidata descifra el mensaje y se busca la frase requerida. Un acierto se confirma con HMAC-SHA256 para descartar falsos positivos.
Open MPI	Cada proceso prueba candidatas con su propia memoria. La base comparte resultados por ronda; la alternativa solicita bloques y envía avisos de hallazgo [4-6].

NIST precisa: *«Each of them transforms data in blocks of 128 bits»*.[1] La longitud del bloque se mantiene aunque las claves del experimento procedan de enteros.

Cómo se mide la mejora

```
S = T_secuencial / T_MPI       E = S / 4
R = intentos / tiempo          G = T_base / T_alternativa
```

S es el speedup y E la eficiencia con cuatro procesos. G compara las dos estrategias: un valor mayor que uno favorece a la alternativa. Cambiar el orden de candidatas también cambia el trabajo hasta el hallazgo; por eso G no mide solo paralelismo.

1. NIST (2023), FIPS 197-upd1, resumen. Cita en inglés; enlace completo en la referencia [1].

2. Archivos, cifrado y validación

El programa aes_archivo carga hasta 4096 bytes, cifra con una clave decimal y recupera el contenido con la misma clave. Lee y escribe bytes para conservar UTF-8 y saltos de línea. Las salidas se crean en modo exclusivo para proteger archivos existentes.

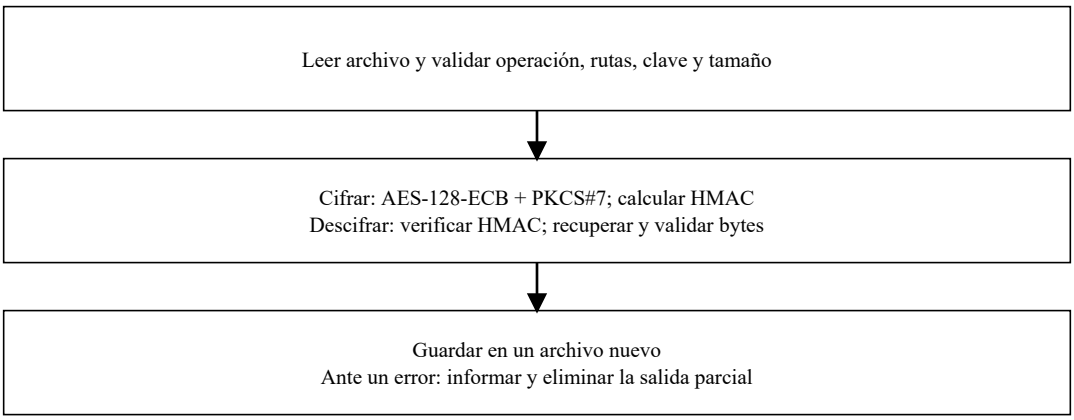


Figura 1. Flujo del inciso 1. Fuente: aes_archivo.c y aes_comun.h.

Formato y claves

AESECB2 guarda cuatro líneas: versión, longitud original, cifrado hexadecimal y etiqueta HMAC-SHA256. La etiqueta cubre versión, longitud y cifrado; la clave MAC se deriva con el dominio Proyecto02/AESECB2/HMAC-SHA256. El archivo se autentica antes de descifrar [8]. El formato GCM del laboratorio se conserva como antecedente separado.

Caso	Clave decimal	Clave AES de 16 bytes (hexadecimal)
a)	123456	0000000000000000000000000001e240
b)	18014398509481984	00000000000000000040000000000000
c)	18014398509481985	00000000000000000040000000000001

Verificación

cmp y SHA-256 comprueban que el archivo recuperado coincide byte a byte con el original. La suite incluye un vector AES-128-ECB, 24 casos de archivos y 75 de búsqueda. Se prueban archivos vacíos, límites de tamaño, claves incorrectas, contenido alterado, rangos extremos, tiempo agotado y hallazgos simultáneos. El registro final terminó con código 0.

```
./proyecto02/build/aes_archivo cifrar entrada.txt salida.aes 123456
./proyecto02/build/aes_archivo descifrar salida.aes recuperado.txt 123456
cmp entrada.txt recuperado.txt
```

3. Estrategia base: lotes estáticos

El proceso 0 valida el archivo y difunde los parámetros y el cifrado. Con cuatro procesos y lotes de B = 65536 candidatas, cada ronda entrega un lote disjunto a cada proceso. La referencia secuencial recorre el mismo intervalo en orden ascendente.

Ronda	P0	P1	P2	P3
0	[0, B)	[B, 2B)	[2B, 3B)	[3B, 4B)
1	[4B, 5B)	[5B, 6B)	[6B, 7B)	[7B, 8B)

```
inicio(p, r) = offset + (4*r + p)*B
fin(p, r) = min(inicio + B, offset + limite)
```

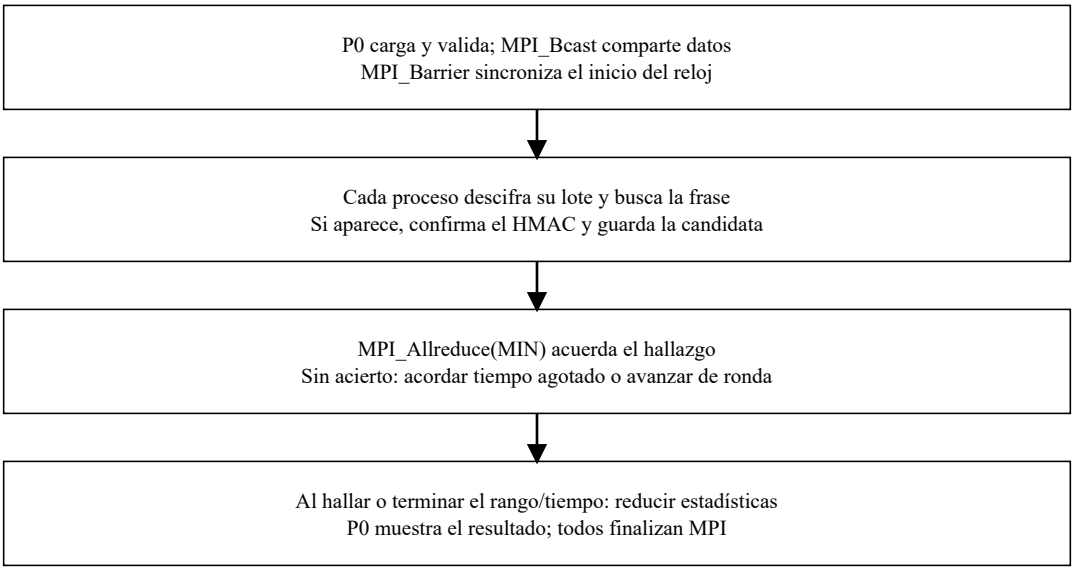


Figura 2. Flujo de la base. Las reducciones mantienen el mismo estado entre procesos [4].

Comportamiento esperado

El reparto requiere poca gestión, pero la parada espera al cierre de la ronda. Para 123456, P1 encuentra la clave en la primera ronda mientras los demás completan sus lotes. El trabajo adicional explica por qué un aumento de claves por segundo no se traduce en la misma reducción del tiempo de búsqueda.

El valor UINT64_MAX indica ausencia de hallazgo. Los rangos que desbordan se rechazan antes de buscar. Los estados son: 0 hallada, 1 entrada/error, 2 rango agotado y 3 tiempo agotado.

4. Alternativa: familias y avisos

La alternativa genera candidatas por familias y asigna sus bloques de forma dinámica. Cada proceso empieza por la familia de su rank y, al agotarla, sigue en orden circular. Las claves objetivo nunca se reciben como argumento de búsqueda.

Familia	Candidatas
F0. Valores bajos	Enteros en [0, 2^24). Incluye 123456.
F1. Patrones	Fechas numéricas, dígitos repetidos, potencias de 2 y 10 con vecinos y constantes. Incluye 2^54 y 2^54 + 1.
F2. ASCII	Cadenas de 1 a 5 caracteres de [a-z0-9], empaquetadas como entero big endian.
F3. Barrido	Intervalo configurable. Por defecto comienza en 2^24 y contiene 2^63 candidatas.

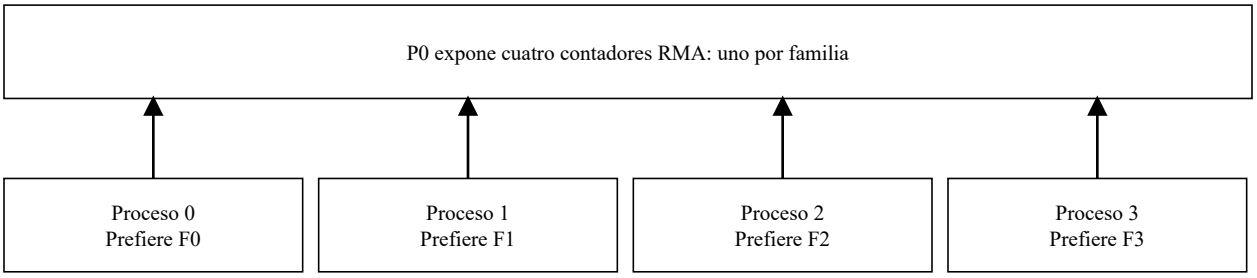


Figura 3. MPI_Fetch_and_op reserva un bloque exclusivo; MPI_Win_flush completa la operación [5].

```
publicar recepción de aviso; sincronizar e iniciar reloj
mientras haya trabajo y no haya aviso ni tiempo agotado:
    elegir familia; reservar bloque con incremento atómico
    para cada candidata del bloque:
        descifrar, buscar frase y confirmar HMAC
        si acierta: enviar clave/familia con MPI_Issend; salir
        cada 4096 intentos: consultar MPI_Test y reloj
    consultar aviso y reloj entre bloques
acordar clave y estado; reducir tiempo e intentos
drenar avisos hasta completar envíos; liberar recursos
```

Cada proceso deja un MPI_Irecv abierto y sondea el aviso de hallazgo [6]. El consenso final resuelve aciertos simultáneos. Los índices no se repiten dentro de una familia, aunque distintas familias pueden producir la misma candidata. El modo -e conserva solo F3 y permite comparar la distribución sin priorizar patrones.

5. Método y resultados

Se registraron 60 ejecuciones el 2026-10-27: cinco por combinación de programa y caso, incluido un control. Las versiones MPI usaron cuatro procesos. Se informa la mediana; el Anexo 2 reúne las muestras y resultados.md conserva su dispersión.

Condición	Configuración registrada
Entrada	Esta es una prueba de proyecto 2; frase: es una prueba de.
Entorno	WSL; Intel(R) Core(TM) Ultra 5 125U. GCC 15.2.0, OpenSSL 3.5.5 y Open MPI 5.0.10; C11 y -O2.
Serie principal	Base/secuencial: [0, 2^56), sin offset cercano a la clave. Lote 65536; corte de 5 s en recorridos largos.
Control	Clave 750123; rango [0, 1000000); lote 4096; alternativa con -e.

Tiempo hasta hallar la clave

Caso / clave	Secuencial (s)	Base MPI (s)	Alternativa (s)
a) 123456	0.080493	0.054261	0.096404
b) 18014398509481984	Sin hallazgo*	Sin hallazgo*	0.071109
c) 18014398509481985	Sin hallazgo*	Sin hallazgo*	0.063716

** En b) y c), secuencial y base finalizaron por tiempo con código 3. El intervalo sí incluye las claves.*

Duración de las búsquedas interrumpidas

Caso / programa	Tiempo (s)	Intentos (mediana)	Claves/s
b) secuencial	5.001534	9,900,032	1,979,846
b) base	5.020116	18,087,936	3,600,815
c) secuencial	5.000482	9,519,104	1,903,562
c) base	5.054247	17,563,648	3,466,635

Los relojes miden la búsqueda: excluyen lectura, lanzamiento MPI y reporte. La alternativa incluye el consenso de clave y excluye el drenaje posterior. El entorno WSL comparte recursos; las cinco muestras describen esta ejecución, sin establecer escalabilidad entre máquinas.

6. Análisis del rendimiento

Comparación medida	S base/sec.	E base	G base/alt.
a) Clave 123456	1.483	37.1 %	0.563
Control: clave 750123	1.900	47.5 %	0.736

Para a), S = 0.080493 / 0.054261 = **1.483**. La base fue más rápida que la secuencial y que la alternativa. Esta última dedica parte del trabajo a otras familias y añade coordinación. Esos costos pueden explicar su mayor tiempo en una búsqueda tan corta.

El control sin heurísticas

Programa	Mediana (s)	Mínimo (s)	Máximo (s)
secuencial	0.460884	0.411778	0.552827
base	0.242585	0.216769	0.266417
alternativa	0.329376	0.283342	0.347526

Con el mismo rango y lote, la base vuelve a obtener el menor tiempo. La asignación dinámica y los avisos anticipados no bastaron para superarla. El número de intentos puede variar por la asignación y el momento de parada; el control compara esas políticas sin el efecto de las familias.

Por qué las claves grandes se encuentran pronto

2^54 y 2^54 + 1 aparecen cerca dentro de la familia de potencias y vecinos. Su ventaja proviene del orden de candidatas. El costo de AES por intento no crece con el valor decimal de la clave; crece el número de intentos necesarios para alcanzarla en un recorrido ascendente.

Proyección del recorrido exhaustivo

Caso	Secuencial (años)	Base MPI (años)
b)	288.3	158.5
c)	299.9	164.7

Se calcula T = (clave + 1) / mediana(claves/s), con tasa constante y año de 365,25 días. Es una extrapolación desde cinco segundos de búsqueda. No se calcula speedup de hallazgo para b) y c), porque faltan tiempos exhaustivos completos.

7. Discusión y conclusiones

Retos y soluciones

La revisión del programa resolvió tres problemas principales: aceptar claves incorrectas con relleno válido, desbordar rangos de 64 bits y coordinar la terminación MPI. Se incorporaron autenticación HMAC, validación de intervalos y consenso de estado con drenaje de avisos. Las regresiones cubren esas rutas.

En WSL con Open MPI 5, el helper evita una ruta problemática de PRRTE y recupera el estado explícito que imprime el programa. Si falta ese estado o vence el timeout externo, la ejecución falla. Las mediciones conservan comandos, intentos, estados y hashes de las fuentes para comprobar su procedencia.

Conclusiones

1. Se completaron el procesamiento de archivos, la búsqueda secuencial y ambas estrategias MPI. El registro final confirma los vectores AES, 24 casos de archivos y 75 de búsqueda.

2. La base obtuvo speedups medidos de 1.483 en 123456 y 1.900 en el control. Fue la opción más rápida en las dos comparaciones sin ventaja por patrones.

3. La alternativa recuperó 2^54 y 2^54 + 1 en medianas de 0.071109 s y 0.063716 s. La priorización explica el hallazgo; los resultados no demuestran una ventaja para claves arbitrarias.

4. Las búsquedas exhaustivas grandes quedaron interrumpidas por costo computacional. El experimento está cerrado con esa limitación documentada; las proyecciones no equivalen a ejecuciones completadas.

Recomendaciones

Para ampliar el estudio conviene medir claves sin estructura, variar el tamaño de lote y repetir con afinidad de procesos y carga controlada. Esas pruebas permitirían evaluar si la asignación dinámica mejora en problemas más largos. Para la entrega actual, deben conservarse las fuentes y evidencias verificadas, junto con el paquete reproducible. ECB y las claves numéricas se limitan al objetivo didáctico.

Referencias bibliográficas

[1] National Institute of Standards and Technology. (2023). *Advanced Encryption Standard (AES), FIPS 197-upd1*.

https://csrc.nist.gov/pubs/fips/197/final

[2] Dworkin, M. (2001). *Recommendation for Block Cipher Modes of Operation: Methods and Techniques. NIST SP 800-38A*.

https://csrc.nist.gov/pubs/sp/800/38/a/final

[3] OpenSSL Project. (s. f.). *EVP_EncryptInit: EVP cipher routines, OpenSSL 3.5*. Secciones de relleno y notas.

https://docs.openssl.org/3.5/man3/EVP_EncryptInit/

[4] Open MPI. (s. f.). *MPI_Allreduce. Documentación de Open MPI 5.0.x.*

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Allreduce.3.html

[5] Open MPI. (s. f.). *MPI_Fetch_and_op. Documentación de Open MPI 5.0.x.*

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Fetch_and_op.3.html

[6] Open MPI. (s. f.). *MPI_Issend. Documentación de Open MPI 5.0.x.*

https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/MPI_Issend.3.html

[8] OpenSSL Project. (s. f.). *HMAC. Documentación de OpenSSL 3.5.*

https://docs.openssl.org/3.5/man3/HMAC/

[7] Universidad del Valle de Guatemala. (2026). *CC3069 - Proyecto 02*. Enunciado y rúbrica del curso Computación Paralela y Distribuida, Ciclo 2. Documento proporcionado con el proyecto.

Los diagramas y cálculos se elaboraron a partir del código y las evidencias del repositorio.

Anexo 1. Catálogo de funciones

A1.1. AES y formato: proyecto02/include/aes_comun.h

Función	Entradas	Salida y funcionamiento
dataset_tag	Candidato, Dataset y etiqueta destino.	Deriva una clave MAC por dominio; calcula HMAC-SHA256 sobre versión, longitud y cifrado.
verify_dataset	Candidato y Dataset.	Recalcula y compara la etiqueta; 1 auténtico, 0 no válido, -1 error de biblioteca.
cipher_length	Longitud original.	Devuelve (longitud/16 + 1)·16; reserva relleno de 1 a 16 bytes.
make_key	Candidato uint64_t y arreglo de 16 bytes.	Llena el arreglo: ocho bytes iniciales en cero y entero en big endian.
parse_number	Cadena y puntero a entero.	1 si es decimal válido y distinto de UINT64_MAX; escribe el valor. 0 ante error.
encrypt_message	Contexto EVP, candidato, texto, longitud y Dataset.	Cifra con AES-128-ECB; llena Dataset, autentica y comprueba la longitud; devuelve éxito.
decrypt_message	Contexto, candidato, Dataset, búfer y puntero de longitud.	1 si HMAC, longitud y relleno son válidos; 0 si fallan; -1 ante error de inicialización/proceso. Escribe texto y longitud.
print_hex	FILE*, bytes y longitud.	Escribe hexadecimal minúsculo y salto de línea; no devuelve estado.
read_hex	FILE*, destino y cantidad de bytes.	Lee pares hexadecimales y exige salto final; 1 válido, 0 inválido.
save_dataset	Ruta y Dataset.	Escribe AESECB2, longitud, cifrado y etiqueta; devuelve éxito de E/S. Utilidad disponible, no usada por aes_archivo.
load_dataset	Ruta y Dataset de destino.	Valida cabecera, longitud, hexadecimal y fin de archivo; llena Dataset y devuelve éxito.

A1.2. Archivos: proyecto02/src/aes_archivo.c

Función	Entradas	Salida y funcionamiento
read_text	Ruta, destino y puntero de longitud.	Lee hasta 4096 bytes; rechaza excedentes o errores; conserva bytes y devuelve éxito.
write_output	Ruta, Dataset, texto y modo.	Abre con wbx; guarda contenedor o texto; elimina salida parcial si falla.
main	argc y argv: operación, entrada, salida, clave.	Coordina validación, EVP y E/S; limpia memoria. Código 0 si termina; 1 si falla.

Anexo 1. Búsqueda y MPI

A1.3. Comunes: proyecto02/include/busqueda.h y busqueda_mpi.h

Función	Entradas	Salida y funcionamiento
init_params	Params*, límite, lote y offset.	Inicializa valores, tiempo sin límite y frase por defecto.
parse_params	argc, argv, Params* y ruta de salida.	Lee -e, -f, -l, -b, -o y -t; rechaza rangos que desbordan y valida límites y un archivo; devuelve 1/0.
contains_phrase	Texto, longitud, frase y tamaño.	Compara ventanas con memcmp; devuelve 1 si contiene la frase.
try_phrase	EVP, candidato, Dataset, destino y frase.	Descifra, busca la frase y confirma con HMAC; 1 acierto, 0 ausente, -1 error EVP.
phrase_fits	Params y Dataset.	Devuelve si la longitud de la frase cabe en la longitud original declarada.
wall_seconds	Sin parámetros.	Tiempo double con CLOCK_MONOTONIC; termina con error si falla el reloj.
print_report	Modo, procesos, clave, intentos, tiempo, estado, EVP, datos y parámetros.	Muestra estado, clave hexadecimal, mensaje, tiempo y rendimiento.
mpi_setup	argc, argv, uso, Params y Dataset.	P0 valida y carga; difunde estado y datos mediante Bcast; todos reciben el mismo estado.
mpi_fail	Mensaje de error.	Escribe en stderr e invoca MPI_Abort para terminar el comunicador.
mpi_finish	Estado local de aplicación.	Consenso MAX, imprime Estado MPI en P0, finaliza y devuelve el estado. El helper WSL recupera ese código.

A1.4. Puntos de entrada de las búsquedas

Archivo / función	Entradas	Salida y funcionamiento
busqueda_secuencial.c / main	Opciones y archivo AES.	Recorre desde offset hasta el fin, prueba frase y mide tiempo. Libera EVP.
busqueda_base.c / main	Opciones y archivo AES; rank y tamaño MPI.	Reparte lotes, reduce el acierto por ronda y obtiene tiempo máximo e intentos totales.
busqueda_alternativa.c / main	Opciones y archivo AES; rank y tamaño MPI.	Crea contadores RMA, asigna bloques, sondea avisos, acuerda resultado y libera recursos.

Códigos de salida de las búsquedas: 0 = clave hallada; 1 = entrada o error; 2 = rango agotado sin hallar; 3 = tiempo agotado. Un aborto MPI puede producir un diagnóstico adicional del lanzador.

Anexo 1. Generadores y scripts

A1.5. Familias: proyecto02/src/busqueda_alternativa.c

Función	Entradas	Salida y funcionamiento
pow_u64	Base y exponente entero.	Producto iterado en uint64_t; calcula potencias usadas por los generadores.
key_patrones	Índice de la familia.	Devuelve fecha, repetición, potencia/vecino o constante según el segmento del índice.
ascii_total	Sin parámetros.	Suma 36^l para l de 1 a 5: cantidad de índices de la familia ASCII.
key_ascii	Índice ASCII.	Determina longitud, descompone en base 36 y empaqueta caracteres como entero.
family_count	Identificador de familia y Params.	Devuelve el número de índices: límite de bajos, patrones, ASCII o barrido.
family_key	Familia, índice y Params.	Selecciona el generador; en el barrido devuelve offset + índice.

A1.6. Automatización de pruebas y mediciones

Módulo / rutinas	Entradas y salidas; propósito
probar_inciso1.py: run, main	run ejecuta y comprueba éxito/error esperado; main crea archivos temporales, verifica bytes y 24 casos.
probar_busqueda.py: run, encrypt, found_key, main	run valida el código con timeout; encrypt genera un caso; found_key extrae la clave; main comprueba 75 casos; Open MPI es obligatorio.
ejecutar_pruebas.py: field, main; save y execute internas	field extrae métricas; execute valida estados; save conserva datos crudos; main mide serie principal y control con metadatos, JSON y tablas.
mpi_utils.py: mpi_prefix, run_command	Selecciona flags de Open MPI según entorno; ejecuta con timeout y termina el grupo completo si se excede.
registrar_pruebas.py: main	Ejecuta make test y guarda stdout, stderr, fecha y código en evidencias/pruebas/.
empaquetar.py: main	Reúne fuentes, ejecutables, informe y evidencias; genera ZIP y comprueba su integridad.

Alcance del catálogo: todas las funciones C de proyecto02 y las rutinas Python que verifican, miden y empaquetan la implementación. El Laboratorio 03 se conserva como antecedente; su API GCM no interviene en las búsquedas ECB del proyecto.

Anexo 1. Librerías y rutinas externas

Librería / API	Entradas, salidas y uso en el proyecto
OpenSSL: EVP_CIPHER_CTX_new / free	Crea un contexto o NULL; free lo libera. Un contexto independiente por proceso.
EVP_aes_128_ecb; EncryptInit/Update/Final	Selecciona cifrador; recibe clave y bytes; produce cifrado, longitudes y estado.
DecryptInit/Update/Final; set_padding	Recibe clave/cifrado y política de relleno; devuelve bytes y estado. Final verifica el relleno si está habilitado.
OPENSSL_cleanse	Puntero y cantidad de bytes; limpia búferes antes de liberar o descartar datos.
MPI_Init / Finalize; Comm_rank / size	Inicializa/finaliza el entorno y consulta identificador y cantidad de procesos.
MPI_Bcast / Barrier / Wtime	Difunde estado y datos desde P0; sincroniza el inicio; devuelve reloj local en segundos.
MPI_Allreduce / Reduce	Recibe valores, tipo y operador; combina MIN de claves, MAX de tiempo/estado y SUM de intentos. Allreduce devuelve a todos; Reduce a P0.
MPI_Win_create / lock_all / unlock_all / free	Expone contadores, abre/cierra época de acceso y libera ventana. Devuelve manejador/estado.
MPI_Fetch_and_op / Win_flush	Recibe incremento, destino y desplazamiento; retorna contador anterior y asegura finalización remota.
MPI_Irecv / Issend / Test	Publica recepción/envíos de clave y familia; devuelve solicitudes. Test consulta finalización y datos recibidos.
MPI_Testall / Cancel / Wait	Progresa y consulta envíos mientras se drenan avisos; cancela/completa la recepción pendiente.
HMAC / EVP_sha256 / CRYPTO_memcmp	Derivación por dominio y etiqueta SHA-256 de 32 bytes; comparación de autenticación sin memcmp ordinario.
MPI_Iprobe / Recv / Abort	Consulta y drena mensajes adicionales; Abort termina el comunicador ante error no recuperable.
C: stdio, stdlib, string, stdint, inttypes, errno	Archivos y E/S, conversión decimal, memoria/cadenas, enteros y formato portable, detección de errores.
POSIX: getopt y clock_gettime	Procesa opciones desde argv y obtiene reloj monotónico. Encabezados unistd.h y time.h.
Python: pathlib, subprocess, tempfile, re, statistics, zipfile	Rutas, procesos, archivos temporales, extracción de métricas, mediana y empaquetado. ReportLab genera este informe.

Dataset conserva longitud, cifrado y etiqueta HMAC. Params contiene rangos, tiempo, modo y frase. La difusión usa MPI_UINT64_T, MPI_INT y caracteres explícitos; no transmite el padding de estructuras nativas.

Anexo 2. Compilación y verificación

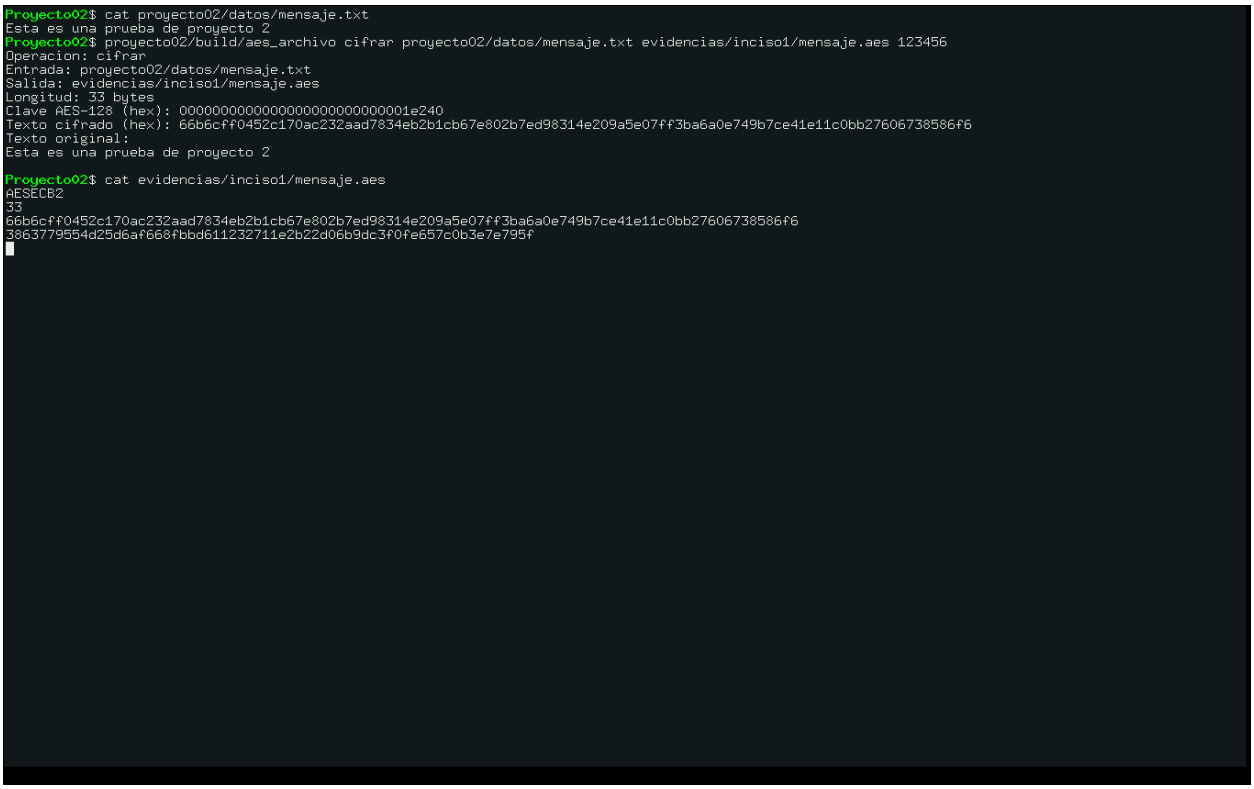
```
Proyecto02$ date '+%Y-%m-%d %H:%M:%S %Z' && uname -sr
2026-10-27 22:26:28 CST
Linux 6.18.33.2-microsoft-standard-WSL2
Proyecto02$ gcc --version | head -1 && mpirun --version | head -1 && openssl version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
mpirun (Open MPI) 5.0.10
OpenSSL 3.5.5 27 Jan 2026 (Library: OpenSSL 3.5.5 27 Jan 2026)
Proyecto02$ make clean >/dev/null && make && make laboratorio | grep -E 'gcc|mpicc'
make -C proyecto02
make[1]: Entering directory '/mnt/c/Users/carlos.estrada/Documents/U/Proyecto-2---Paralela/proyecto02'
mkdir -p build
gcc -Iinclude -D_POSIX_C_SOURCE=200809L -std=c11 -O2 -Wall -Wextra -Wpedantic src/aes_archivo.c -o build/aes_archivo  -lcrypto
gcc -Iinclude -D_POSIX_C_SOURCE=200809L -std=c11 -O2 -Wall -Wextra -Wpedantic src/busqueda_secuencial.c -o build/busqueda_secuencial  -lcrypto
mpicc -Iinclude -D_POSIX_C_SOURCE=200809L -std=c11 -O2 -Wall -Wextra -Wpedantic src/busqueda_base.c -o build/busqueda_base  -lcrypto
mpicc -Iinclude -D_POSIX_C_SOURCE=200809L -std=c11 -O2 -Wall -Wextra -Wpedantic src/busqueda_alternativa.c -o build/busqueda_alternativa  -lcrypto
make[1]: Leaving directory '/mnt/c/Users/carlos.estrada/Documents/U/Proyecto-2---Paralela/proyecto02'
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_secuencial.c -o busqueda_clave_aes_secuencial -lcrypto
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_mejorada.c -o busqueda_clave_aes_mejorada -lcrypto
mpicc -std=c11 -O2 -Wall -Wextra -Wpedantic busqueda_clave_aes_mpi.c -o busqueda_clave_aes_mpi -lcrypto
Proyecto02$ ls proyecto02/build
aes_archivo
busqueda_alternativa
busqueda_base
busqueda_secuencial
```

A2.1. Compilación en WSL de los ejecutables del proyecto y del laboratorio. Transcripción: inciso1/compilacion.txt.

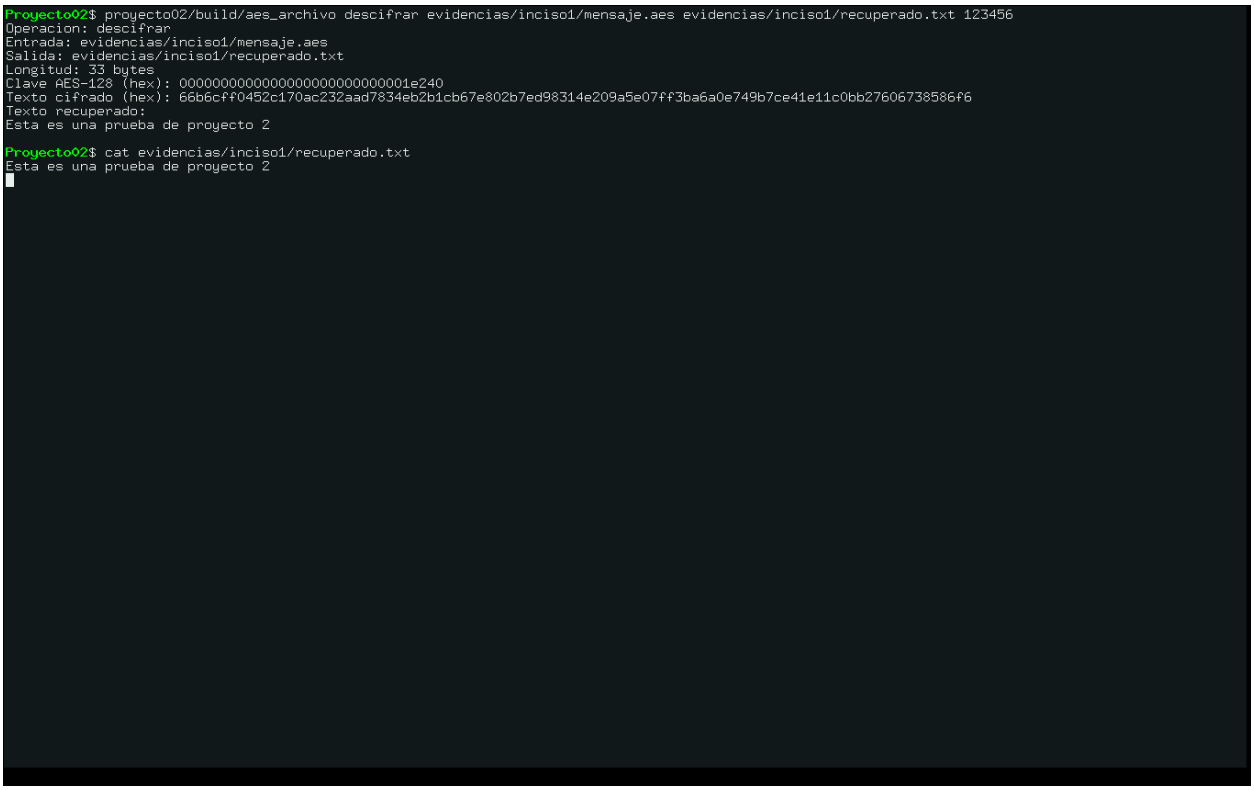
```
Proyecto02$ cmp proyecto02/datos/mensaje.txt evidencias/inciso1/recuperado.txt && echo 'Archivos identicos'
Archivos identicos
Proyecto02$ sha256sum proyecto02/datos/mensaje.txt evidencias/inciso1/recuperado.txt
ee3baca1bef803f13ce5c3a754d3209ac15d2f96d679e00f1d882fed7732d257  proyecto02/datos/mensaje.txt
ee3baca1bef803f13ce5c3a754d3209ac15d2f96d679e00f1d882fed7732d257  evidencias/inciso1/recuperado.txt
Proyecto02$ proyecto02/build/aes_archivo descifrar evidencias/inciso1/mensaje.aes evidencias/inciso1/no_crear.txt 44
Clave incorrecta o contenido alterado.
[codigo de salida: 1]
Proyecto02$ test ! -e evidencias/inciso1/no_crear.txt && echo 'Clave 44 rechazada; no se genero salida'
Clave 44 rechazada; no se genero salida
Proyecto02$ python3 proyecto02/tests/probar_inciso1.py
PASS: 24 casos del inciso 1; archivos identicos byte a byte,
limites, autenticacion HMAC, errores y formatos ajenos (AES-128-ECB).
```

A2.2. Archivos idénticos, rechazo de clave incorrecta y pruebas de archivos. Transcripción: inciso1/verificacion.txt.

Anexo 2. Cifrado y recuperación

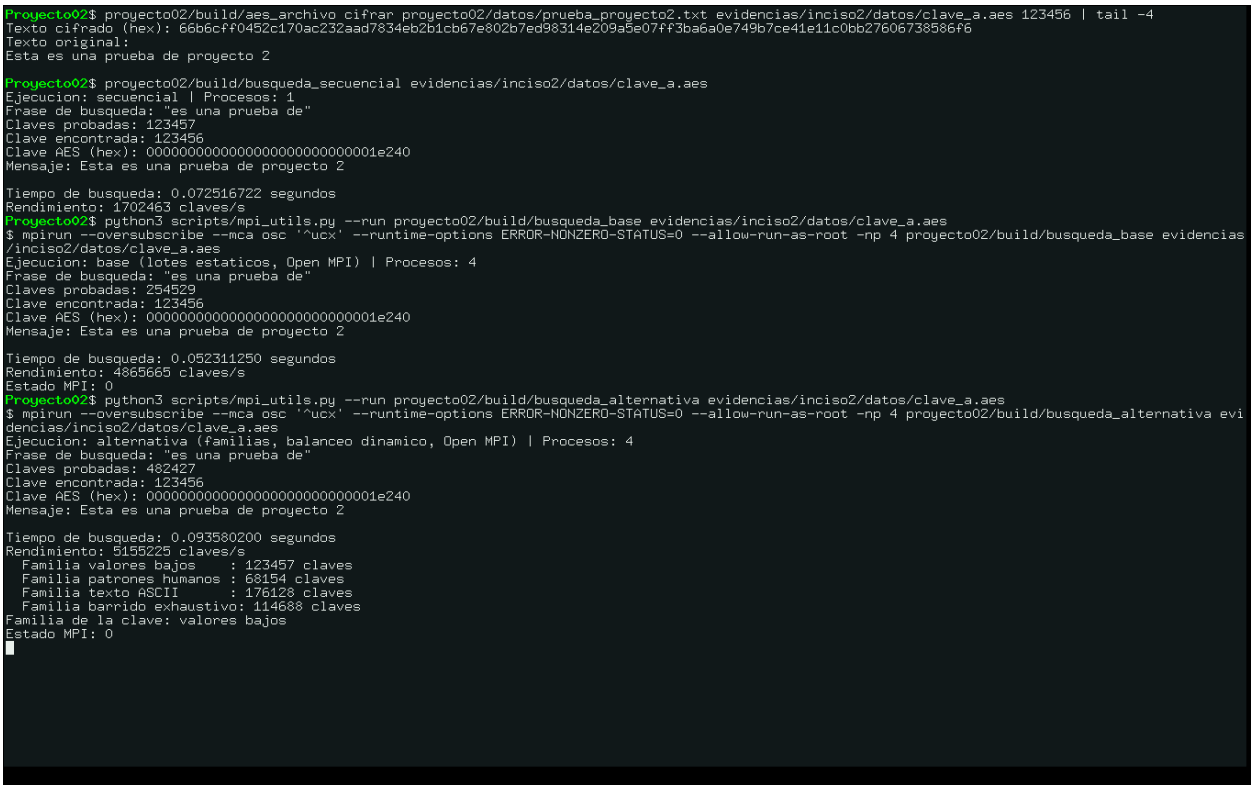


A2.3. Texto original, clave 123456 y cifrado hexadecimal. Transcripción: inciso1/cifrado.txt.



A2.4. Recuperación del mensaje con la misma clave. Transcripción: inciso1/descifrado.txt.

Anexo 2. Caso a): clave 123456



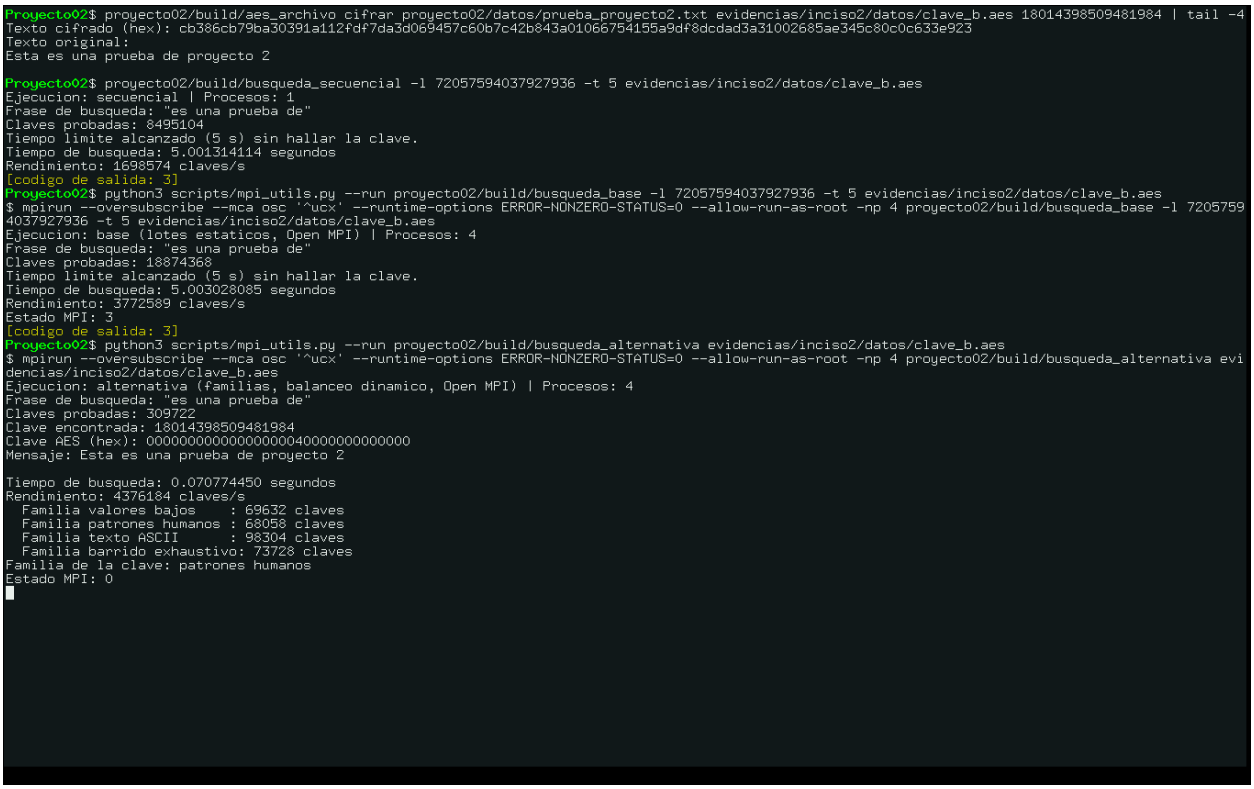
A2.5. Ejecución de las tres versiones. Transcripción: inciso2/clave_a.txt.

Las tres versiones recuperan 123456 y muestran el mensaje requerido. Los tiempos de esta captura pertenecen a una sesión de demostración; las comparaciones del cuerpo usan las cinco muestras de la serie experimental.

Procedencia de las capturas

Las siete imágenes son capturas de sesiones xterm con Xvfb. Sus comandos y salidas se conservan en archivos .txt; capturas.json registra sus hashes. La serie de medianas tiene su propia bitácora y se mantiene separada.

Anexo 2. Caso b): clave 18014398509481984



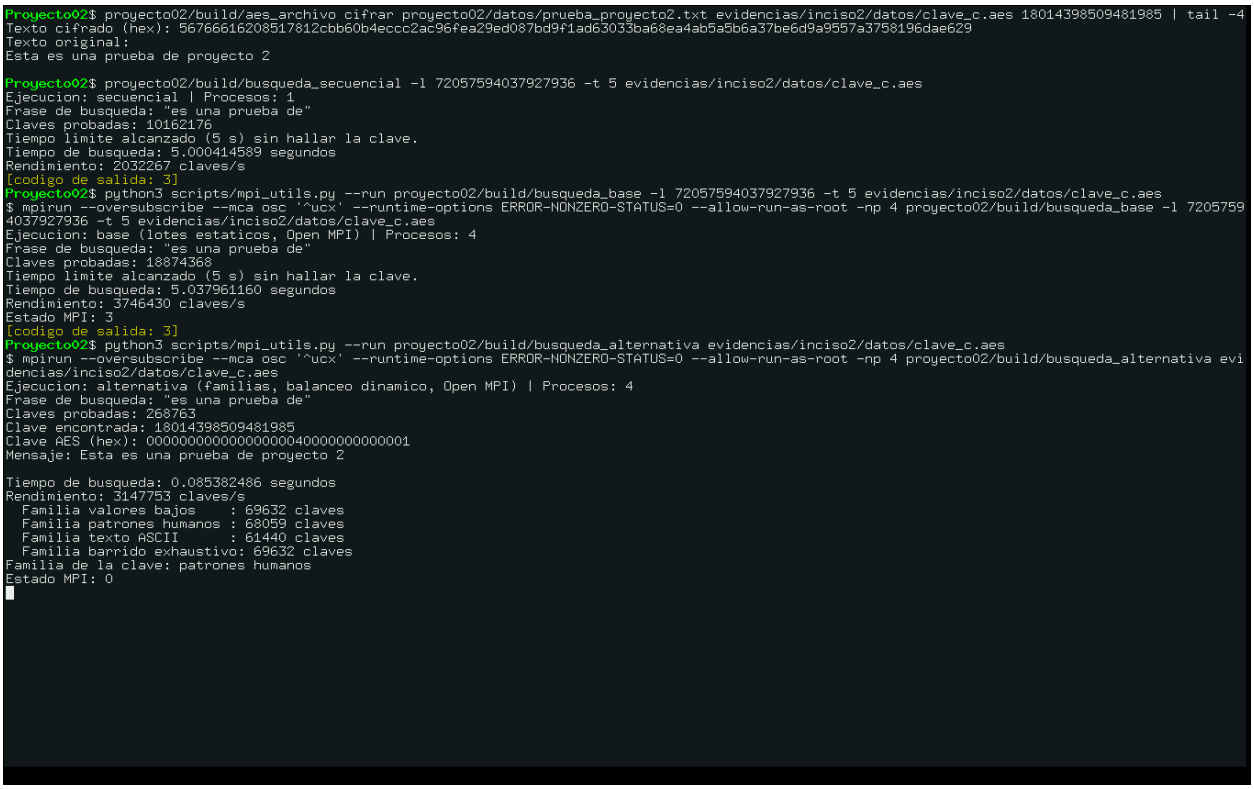
A2.6. Ejecución de las tres versiones. Transcripción: inciso2/clave_b.txt.

La secuencial y la base detienen el recorrido [0, 2^56) tras el corte de 5 segundos, sin hallar la clave. La alternativa la recupera en la familia de patrones, sin recibirla como parámetro.

Procedencia de las capturas

Las siete imágenes son capturas de sesiones xterm con Xvfb. Sus comandos y salidas se conservan en archivos .txt; capturas.json registra sus hashes. La serie de medianas tiene su propia bitácora y se mantiene separada.

Anexo 2. Caso c): clave 18014398509481985



A2.7. Ejecución de las tres versiones. Transcripción: inciso2/clave_c.txt.

La secuencial y la base detienen el recorrido [0, 2^56) tras el corte de 5 segundos, sin hallar la clave. La alternativa la recupera en la familia de patrones, sin recibirla como parámetro.

Procedencia de las capturas

Las siete imágenes son capturas de sesiones xterm con Xvfb. Sus comandos y salidas se conservan en archivos .txt; capturas.json registra sus hashes. La serie de medianas tiene su propia bitácora y se mantiene separada.

Anexo 2. Bitácora y reproducción

Tiempos individuales en segundos. Cada fila reúne cinco ejecuciones; en las filas marcadas con * se alcanzó el corte sin hallazgo. Los intentos, tasas, estados, comandos y dispersión se conservan en resultados.json y resultados.md.

Caso / programa	1	2	3	4	5	Mediana
a / secuencial	0.0626	0.0805	0.0734	0.0863	0.0882	0.0805
a / base	0.0543	0.0547	0.0553	0.0504	0.0529	0.0543
a / alternativa	0.1014	0.0918	0.0964	0.0856	0.1226	0.0964
b / secuencial *	5.0016	5.0004	5.0015	5.0003	5.0017	5.0015
b / base *	5.0077	5.0201	5.0233	5.0123	5.0331	5.0201
b / alternativa	0.0722	0.0765	0.0711	0.0660	0.0565	0.0711
c / secuencial *	5.0007	5.0002	5.0003	5.0005	5.0011	5.0005
c / base *	5.0542	5.0541	5.0533	5.0647	5.0665	5.0542
c / alternativa	0.0767	0.0839	0.0535	0.0585	0.0637	0.0637
control / secuencial	0.5528	0.4667	0.4309	0.4118	0.4609	0.4609
control / base	0.2168	0.2334	0.2426	0.2664	0.2591	0.2426
control / alternativa	0.2833	0.2920	0.3408	0.3294	0.3475	0.3294

Reproducir en Ubuntu o WSL

```
sudo apt install build-essential libssl-dev libopenmpi-dev openmpi-bin python3
make
make test
python3 proyecto02/tests/ejecutar_pruebas.py 5 5
```

Para ejecutar MPI en el entorno WSL registrado, usar el helper del repositorio:

```
python3 scripts/mpi_utils.py --run proyecto02/build/busqueda_base \
  -l 72057594037927936 -t 5 evidencias/inciso2/datos/clave_b.aes
```

La entrega incluye fuentes, ejecutables Linux, datos cifrados, evidencias e informe. scripts/empaquetar.py comprueba su procedencia y crea el manifiesto SHA-256; scripts/verificar_entrega.py --ejecutar recompila y prueba una extracción nueva. Las instrucciones para regenerar el PDF están en README.md.